

End-to-End Machine Learning Project

Part 3: Selecting, Fine-Tuning, and Deploying a Model

Resana — Internship Report

This document continues the working reference I’ve been keeping during my internship at Resana, based on Chapter 2 of *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (Aurélien Géron, 3rd ed.). By this point the California housing dataset has already been explored and run through the **preprocessing** pipeline built earlier (ratio features, log-transforms, geographic cluster similarity, and one-hot encoding for the categorical column) – so this part picks up right where that left off: training and comparing baseline models, getting a trustworthy estimate of generalization error with cross-validation, fine-tuning the most promising model, inspecting what it actually learned, evaluating it on the test set, and finally packaging and shipping it. All numbers below are taken directly from my own notebook run on the housing dataset, not from the book’s example.

Contents

1	Select and Train a Model	1
1.1	Train and Evaluate on the Training Set	1
1.2	Better Evaluation Using Cross-Validation	2
2	Fine-Tune Your Model	3
2.1	Grid Search	3
2.2	Randomized Search	4
2.3	Ensemble Methods	4
2.4	Analyzing the Best Models and Their Errors	5
2.5	Evaluate Your System on the Test Set	5
3	Launch, Monitor, and Maintain Your System	6

1 Select and Train a Model

1.1 Train and Evaluate on the Training Set

With the data cleaned up and the preprocessing pipeline ready, the obvious first move is the simplest possible baseline: a plain linear regression, wrapped together with the preprocessing pipeline so the two train as one unit.

```
1 from sklearn.linear_model import LinearRegression
2
3 lin_reg = make_pipeline(preprocessing, LinearRegression())
4 lin_reg.fit(housing, housing_labels)
```

Comparing the first few predictions against the real labels shows the model is in the right ballpark but far from precise – some districts are off by tens of thousands of dollars. To put a single number on this, I measured the RMSE on the training set itself:

```
1 from sklearn.metrics import root_mean_squared_error
```

```

2         lin_rmse = root_mean_squared_error(housing_labels, housing_predictions
3         )

```

Deprecation note. Older code (and the book’s earlier editions) computed RMSE via `mean_squared_error(y_true, y_pred, squared=False)`. That `squared` argument is deprecated in recent Scikit-Learn versions – `root_mean_squared_error` is the direct replacement.

This gave `lin_rmse` $\approx$ \$69,123. Since most district values fall between \$120,000 and \$265,000, a typical prediction error of \$69k is not very satisfying – a textbook case of *underfitting*: either the features don’t carry enough signal for a linear model, or the model itself is simply too constrained for the underlying pattern.

As a next step I tried a `DecisionTreeRegressor`, a model capable of finding much more complex, nonlinear relationships in the data:

```

1         from sklearn.tree import DecisionTreeRegressor
2
3         tree_reg = make_pipeline(preprocessing, DecisionTreeRegressor(
4         random_state=42))
5         tree_reg.fit(housing, housing_labels)

```

Evaluated the same way, this model scored `tree_rmse` = 0.0 – literally zero error. That is not a sign of a great model; it’s a sign that the tree simply memorized the training set instead of learning a pattern that generalizes. Evaluating a model on the exact same data it was trained on is close to meaningless, and touching the test set this early would be a mistake too – it needs to stay untouched until I’m confident enough in a model to commit to it. What’s needed instead is a way to evaluate on data the model hasn’t seen, without spending the test set.

1.2 Better Evaluation Using Cross-Validation

Scikit-Learn’s k -fold cross-validation solves exactly this: it randomly splits the training set into k non-overlapping folds, then trains and evaluates the model k times, each time holding out a different fold for validation and training on the rest.

```

1         from sklearn.model_selection import cross_val_score
2
3         tree_rmse = -cross_val_score(tree_reg, housing, housing_labels,
4         scoring="neg_root_mean_squared_error",
5         cv=10)

```

Caution. Scikit-Learn’s cross-validation tooling expects a utility function (*higher is better*), not a cost function, so `scoring` here is actually the negative of RMSE. That’s why the sign has to be flipped back with the leading `-`.

Summarizing the 10 fold scores with `pd.Series(tree_rmse).describe()` gave a mean RMSE of about \$67,647 with a standard deviation of about \$2,619. So the decision tree, despite scoring a perfect 0 on the training set, performs roughly as poorly as the linear model once evaluated on data it hasn’t memorized – and cross-validation now also tells me *how* precise that estimate is (the standard deviation), which a single train/test split never would. The gap between a training error

of 0 and a cross-validation error of \$67.6k is a clean, quantitative confirmation of the overfitting I suspected above.

Next up, a `RandomForestRegressor` – an *ensemble* method that trains many decision trees, each on a random subset of features, and averages their predictions:

```
1 from sklearn.ensemble import RandomForestRegressor
2
3 forest_reg = make_pipeline(preprocessing,
4                             RandomForestRegressor(random_state=42))
5 forest_rmse = -cross_val_score(forest_reg, housing, housing_labels,
6                                 scoring="neg_root_mean_squared_error", cv=10)
```

This came out to a mean RMSE of about \$47,313 (std $\approx$ \$2,389) – clearly the best of the three so far, and a big jump over both the linear model and the single decision tree. Random forests are still not immune to overfitting, though: the book’s own worked example on this same setup shows a training RMSE around \$17,474 for a random forest, i.e. well below its cross-validation score, meaning there’s still a real (if much smaller) gap between training and validation performance. Still, with several reasonable candidates compared, the random forest is clearly the most promising one to carry forward and fine-tune – so that’s the shortlist for the next section.

2 Fine-Tune Your Model

2.1 Grid Search

Rather than hand-tweaking hyperparameters, Scikit-Learn’s `GridSearchCV` can search a defined grid of values automatically, cross-validating every combination. To target hyperparameters that live *inside* the preprocessing pipeline (like the number of geographic clusters) as well as the model’s own hyperparameters, the pipeline steps need names – which means switching from `make_pipeline` to `Pipeline`:

```
1 from sklearn.model_selection import GridSearchCV
2 from sklearn.pipeline import Pipeline
3
4 full_pipeline = Pipeline([
5     ("preprocessing", preprocessing),
6     ("random_forest", RandomForestRegressor(random_state=42)),
7 ])
8 param_grid = [
9     {'preprocessing__geo__n_clusters': [5, 8, 10],
10      'random_forest__max_features': [4, 6, 8]},
11     {'preprocessing__geo__n_clusters': [10, 15],
12      'random_forest__max_features': [6, 8, 10]},
13 ]
14
15 grid_search = GridSearchCV(full_pipeline, param_grid, cv=3,
16                             scoring='neg_root_mean_squared_error')
17 grid_search.fit(housing, housing_labels)
```

The double-underscore naming (`preprocessing__geo__n_clusters`) is how Scikit-Learn reaches through nested steps: go into the "preprocessing" step, find the sub-transformer named "geo"

inside it, and set its `n_clusters` parameter. Same idea for `random_forest__max_features`, which just refers to the `RandomForestRegressor` step directly.

Tip. *Tuning preprocessing hyperparameters together with the model's is worthwhile because they often interact – e.g. more clusters may call for a higher `max_features` too. If any preprocessing step is expensive to recompute, the pipeline's `memory` hyperparameter can point to a caching directory so identical steps aren't rerun for every combination that shares them.*

With two dictionaries of 9 and 6 combinations respectively (15 total) and `cv=3`, this amounts to 45 rounds of training in total. The winning combination was:

```
1 >>> grid_search.best_params_  
2 {'preprocessing__geo__n_clusters': 15, 'random_forest__max_features':  
   6}
```

which corresponds to a mean cross-validation RMSE of about \$44,366 – already a solid improvement over the untuned forest's \$47,313. Sorting `grid_search.cv_results_` by score surfaces something worth flagging: two of the top rows (`n_clusters=10`, `max_features=6`) are tied with the exact same score, down to the decimal. That's not a coincidence – it's because my two `param_grid` dictionaries overlap at that one combination (10 appears in both `n_clusters` lists, 6 in both `max_features` lists), so `GridSearchCV` trained and scored it twice without me noticing. It didn't change the outcome here, but it's wasted compute, and worth double-checking for next time – whenever `param_grid` is a list of several dictionaries, it's worth checking they don't overlap.

2.2 Randomized Search

Grid search is fine for a handful of values, but it doesn't scale: exploring a wider range means testing every combination, which grows very fast. `RandomizedSearchCV` instead samples a fixed number of combinations from a distribution, which means the size of the search space no longer has to match the compute budget:

```
1 from sklearn.model_selection import RandomizedSearchCV  
2 from scipy.stats import randint  
3  
4 param_distributions = {'preprocessing__geo__n_clusters': randint(low=3,  
5                           high=50),  
6                           'random_forest__max_features': randint(low=2, high=20)}  
7  
8 rnd_search = RandomizedSearchCV(  
9     full_pipeline, param_distributions=param_distributions, n_iter=10, cv=3,  
10    scoring='neg_root_mean_squared_error', random_state=42)  
11  
    rnd_search.fit(housing, housing_labels)
```

Even with only 10 iterations (versus grid search's 15 combinations), this explored a much wider range of `n_clusters` (up to 50, instead of maxing out at 15) and found a better solution:

```
1 >>> rnd_search.best_params_  
2 {'preprocessing__geo__n_clusters': 45, 'random_forest__max_features':  
   9}
```

with a mean cross-validation RMSE of about \$42,634 – noticeably better than grid search’s \$44,366, and with fewer total fits. That’s the practical case for randomized search: it isn’t guaranteed to beat a grid search over the same set of values, but it lets you search a much larger space for a fixed, predictable amount of compute. (The book also mentions `HalvingGridSearchCV` / `HalvingRandomSearchCV` as a further refinement – they screen candidates cheaply on a small subset of the data first, then only spend full resources on the survivors in later rounds.)

2.3 Ensemble Methods

Beyond tuning a single model, a further way to squeeze out performance is to combine several of the best-performing models into one *ensemble* – the same idea a random forest already applies internally, just one level up. Different model types tend to make different kinds of errors, so averaging their predictions (e.g. a fine-tuned random forest together with a fine-tuned *k*-nearest-neighbors regressor) often outperforms any single model in the group. I didn’t build one for this project, but it’s a natural next step if further gains were needed.

2.4 Analyzing the Best Models and Their Errors

A trained `RandomForestRegressor` can report how much each input feature contributed to its predictions, which is a good way to sanity-check the model (and the feature engineering that fed it):

```
1 final_model = rnd_search.best_estimator_
2 feature_importances_ = final_model["random_forest"].
   feature_importances_
3
4 sorted(zip(feature_importances_,
5 final_model["preprocessing"].get_feature_names_out()),
6 reverse=True)
```

The full list runs to dozens of entries (mostly the individual geographic cluster-similarity features), but the top of the ranking is where the story is:

Feature	Importance
log__median_income	0.181
cat__ocean_proximity_INLAND	0.082
bedrooms__ratio	0.072
rooms_per_house__ratio	0.058
people_per_house__ratio	0.050
geo__Clusters 2 similarity	0.032
geo__Clusters 43 similarity	0.028

`median_income` dominates, by a clear margin – consistent with what the correlation analysis already hinted at earlier in the project. Encouragingly, the three *engineered* ratio features (bedrooms, rooms-per-house, people-per-house) all rank above almost every individual geographic cluster, which suggests that feature engineering step was worth doing. At the opposite end of the list, a handful of features (e.g. the rarer `ocean_proximity` categories, or clusters far from any real geographic pattern) contribute next to nothing.

Tip. `sklearn.feature_selection.SelectFromModel` can automate this: after fitting, it looks at `feature_importances_` and keeps only the most useful features, so the low-value ones above could

be dropped programmatically instead of by hand.

2.5 Evaluate Your System on the Test Set

With a fine-tuned model finally chosen, this is the one and only point where the test set gets touched:

```
1 X_test = strat_test_set.drop("median_house_value", axis=1)
2 y_test = strat_test_set["median_house_value"].copy()
3
4 final_predictions = final_model.predict(X_test)
5 final_rmse = root_mean_squared_error(y_test, final_predictions)
```

This gave `final_rmse` $\approx$ \$39,565 – actually a bit better than the \$42,634 cross-validation estimate. That can happen by chance, but it’s not something to chase: tweaking hyperparameters afterwards to make the test-set number look even better would just mean overfitting to the test set itself, at which point it stops being a trustworthy estimate of real-world performance.

Since a single number doesn’t say much about how *precise* that estimate is, I also computed a 95% confidence interval for the generalization error:

```
1 from scipy import stats
2
3 confidence = 0.95
4 squared_errors = (final_predictions - y_test) ** 2
5 np.sqrt(stats.t.interval(confidence, len(squared_errors) - 1,
6 loc=squared_errors.mean(),
7 scale=stats.sem(squared_errors)))
```

which came out to roughly \$[37,712, 41,336]. In other words, I can say with 95% confidence that the model’s true generalization error falls somewhere in that \$3.6k-wide band – a single point estimate of \$39,565 on its own wouldn’t have conveyed how tight (or loose) that estimate actually is.

3 Launch, Monitor, and Maintain Your System

Getting a model this far is not the finish line – it still has to be packaged, shipped, and kept healthy in production. The simplest deployment path is to save the trained pipeline and load it wherever it needs to run:

```
1 import joblib
2
3 joblib.dump(final_model, "my_california_housing_model.pkl")
```

Since `final_model` is the *entire* pipeline – preprocessing and all, not just the bare `RandomForestRegressor` – calling `.predict()` on raw, unprocessed input automatically runs it through the same preprocessing before making a prediction. One catch: `joblib` only serializes data and parameters, not code. Any custom classes or functions the pipeline depends on (like the `ClusterSimilarity` transformer built earlier for the geo features) have to be defined or imported again in whatever environment loads the model – otherwise unpickling it will fail.

Tip. *It's worth saving every model tried along the way, not just the final one – together with its cross-validation scores and, ideally, its actual predictions on the validation set. That makes it easy to come back later, compare error types across model versions, or roll back if a newer model turns out worse.*

Depending on the use case, this saved pipeline can be loaded directly inside the application, wrapped in a separate web service (e.g. a REST API the app calls over the network), or deployed on a managed cloud platform (like Vertex AI) that takes care of scaling and load balancing on its own.

Shipping the model isn't the end, either – performance tends to decay slowly over time as the real world drifts away from what the training data captured, a phenomenon generally called *model rot* or *data drift*. Catching this requires ongoing monitoring, either through a downstream business metric (e.g. a drop in the click-through or sales rate of recommended products) or through periodic human review of a sample of the model's predictions. Ideally the retraining loop itself is automated: fresh data gets collected, a new model is trained and tuned on it, and it only replaces the live model if it actually beats it on an up-to-date test set. It's also worth monitoring the *input* data quality on its own – a spike in missing values, a statistical shift away from the training distribution, or entirely new categories showing up are all early warning signs, often before the output quality visibly drops. Finally, keeping backups of every model and every dataset version is what makes a quick rollback possible if something does go wrong.

With the model trained, tuned, evaluated, and now deployed with a monitoring plan in place, the end-to-end workflow this project set out to build – starting from raw housing data all the way to a served prediction – is complete.